Web Scraping with BeautifulSoup

Module 18 Study Notes • Custom Data Collection

1. Why Web Scraping?

In the previous module, we fetched data seamlessly using APIs. However, many websites do not offer an API because they do not want their data to be freely downloaded. When an API is unavailable, your only option to collect data is **Web Scraping**—writing a script that reads the raw HTML of a website and extracts the relevant information.

2. Bypassing Server Restrictions (HTTP 403)

If you attempt to scrape a website using Python's `requests.get()`, you may receive a `403 Forbidden` error. This happens because the website's server detects that a Python "bot" is making the request, rather than a human using a web browser.

> **The Fix (Spoofing Headers):** You can trick the server into thinking you are a human by passing a **User-Agent** header in your request. This tells the server that the request is coming from a standard web browser like Google Chrome or Mozilla Firefox.
>
> ```
> headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)...'}
> response = requests.get(url, headers=headers)
> ```

3. Parsing HTML with BeautifulSoup

Once you successfully fetch the webpage's HTML text, you must pass it to a parsing library to make it searchable. The industry standard library for this is `BeautifulSoup`.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, 'lxml')
```

The `'lxml'` argument specifies the parser engine, which is highly efficient for navigating HTML DOM trees.

4. The Two-Step Extraction Strategy

A common mistake beginners make is extracting all titles, all ratings, and all reviews independently. If a single rating is missing on the webpage, your arrays will misalign, ruining the dataset. Instead, use the robust **Two-Step Container Strategy**:

Step A: Isolate the Containers

First, find the HTML element (usually a `<div>`) that wraps an entire single entity (e.g., one complete company block). Use `find_all()` to grab all these containers.

```
containers = soup.find_all('div', class_='company-content-wrapper')
```

Step B: Loop and Extract Internally

Next, loop through these containers. Inside the loop, extract the specific details *only* from that single container using `.find()`. Use `.text.strip()` to pull the clean text out of the HTML tags, removing any invisible whitespace/formatting characters.

```
for item in containers:
    name = item.find('h2').text.strip()
    rating = item.find('p', class_='rating').text.strip()
```

5. Automating Pagination

Just like APIs, websites span multiple pages. To scrape an entire catalog (e.g., 300 pages of companies):

- Identify how the URL changes when you click "Next Page" (e.g., `page=1`, `page=2`).
- Wrap your scraping logic in a `for` loop.
- Dynamically format the URL using the loop index: `url = 'https://site.com/companies?page={}'.format(i)`
- Append the extracted data into a Pandas DataFrame and export it as a CSV.